

Cascading Style Sheets

CSS stands for **Cascading Style Sheets**.

It is a language used to **style and design web pages** — meaning it controls how HTML elements look on the screen.

Simple Explanation

- **HTML** = structure (what content exists)
- **CSS** = appearance (how it looks)
- **JavaScript** = behavior (how it works)

Think of a website like a house:

- HTML → walls and rooms
- CSS → paint, colors, furniture, layout
- JavaScript → electricity and automation

What CSS Does

CSS controls things like:

- Colors
 - Fonts & text size
 - Spacing & alignment
 - Layout (grid, columns, positioning)
 - Backgrounds & images
 - Animations & transitions
 - Responsive design (mobile vs desktop view)
-

Basic Example

HTML

```
<h1>Hello World</h1>
```

CSS

```
h1 {  
  color: blue;  
  font-size: 40px;  
  text-align: center;  
}
```

Result: The text becomes **blue**, **large**, and **centered**.

Why it's called “Cascading”

“Cascading” means styles are applied based on **priority rules**:

- Inline styles override others
- Specific selectors override general ones

- Later rules can override earlier ones

CSS decides which style “wins.”

Types of CSS Usage

1. Inline CSS

```
<h1 style="color:red;">Hello</h1>
```

2. Internal CSS

```
<style>
h1 { color:red; }
</style>
```

3. External CSS (Best Practice)

```
<link rel="stylesheet" href="styles.css">
```

Why CSS is Important

Without CSS:

- Websites look plain and unorganized
- No responsive mobile layouts
- Poor user experience

With CSS:

- Modern UI/UX
 - Professional design
 - Cross-device compatibility
-

If you want, I can also explain **CSS in 5 minutes for beginners**, or show how CSS works with **React / React Native** (useful for your app development).

The **main parts of CSS** are the pieces that make up a **CSS rule** (also called a CSS rule set).

Here is the basic structure:

```
selector {
  property: value;
}
```

1. Selector

The **selector** tells CSS **which HTML element** to style.

Example:

```
h1 {  
  color: blue;  
}
```

`h1` is the selector — it targets all `<h1>` elements.

Common selectors:

- Element selector → `p`, `h1`, `div`
 - Class selector → `.box`
 - ID selector → `#header`
 - Universal selector → `*`
-

2. Declaration Block

Everything inside `{ }` is called the **declaration block**.

```
{  
  color: blue;  
  font-size: 20px;  
}
```

It contains one or more styling instructions.

3. Property

A **property** is *what you want to change*.

Examples:

- `color`
- `font-size`
- `background`
- `margin`
- `width`

Example:

```
color: blue;
```

`color` is the property.

4. Value

The **value** defines *how the property should look*.

Example:

color: blue;

blue is the value.

More examples:

font-size: 18px;
background-color: red;
width: 50%;

Full Example (All Parts Together)

```
p {  
  color: green;  
  font-size: 16px;  
}
```

Part	Meaning
p	Selector
{ }	Declaration block
color	Property
green	Value
font-size:16px;	Declaration

Quick Memory Trick

Selector → Property → Value

“WHAT element → WHAT change → HOW it looks”

If you want, I can next explain the **5 most important CSS concepts beginners must learn** (box model, flexbox, positioning, etc.) — these are what actually make layouts work.

CSS has **hundreds of properties** (300+ in modern CSS).

So instead of listing randomly, the best way is to group them by **category** — this is how developers actually learn and use CSS.

Below is a **complete structured list of the MAIN CSS properties**, clearly explained with examples.

1. Text & Font Properties

These control how text looks.

color

Sets text color.

```
p {  
  color: red;  
}
```

font-size

Controls text size.

```
p {  
  font-size: 18px;  
}
```

font-family

Changes font style.

```
p {  
  font-family: Arial, sans-serif;  
}
```

font-weight

Controls thickness of text.

```
p {  
  font-weight: bold;  
}
```

Values: `normal`, `bold`, `100–900`

text-align

Aligns text horizontally.

```
p {  
  text-align: center;  
}
```

Values: `left`, `right`, `center`, `justify`

text-decoration

Adds/removes underline or line-through.

```
a {  
  text-decoration: none;  
}
```

line-height

Controls spacing between lines.

```
p {  
  line-height: 1.6;
```

```
}
```

letter-spacing

Space between letters.

```
h1 {  
  letter-spacing: 2px;  
}
```

text-transform

Changes letter case.

```
h1 {  
  text-transform: uppercase;  
}
```

2. Box Model Properties

Every HTML element is a box.

width / height

Sets element size.

```
div {  
  width: 300px;  
  height: 200px;  
}
```

margin

Space **outside** the element.

```
div {  
  margin: 20px;  
}
```

padding

Space **inside** the element.

```
div {  
  padding: 15px;  
}
```

border

Adds border around element.

```
div {  
  border: 2px solid black;  
}
```

box-sizing

Controls size calculation.

```
div {  
  box-sizing: border-box;  
}
```

3. Background Properties

background-color

Sets background color.

```
div {  
  background-color: yellow;  
}
```

background-image

Adds image background.

```
div {  
  background-image: url("image.jpg");  
}
```

background-size

Controls image size.

```
div {  
  background-size: cover;  
}
```

background-position

Image placement.

```
div {  
  background-position: center;  
}
```

background-repeat

Repeating behavior.

```
div {  
  background-repeat: no-repeat;  
}
```

4. Layout & Display Properties

display

Defines layout behavior.

```
div {  
  display: flex;  
}
```

Common values:

- `block`
 - `inline`
 - `inline-block`
 - `flex`
 - `grid`
 - `none`
-

position

Controls positioning method.

```
div {  
  position: relative;  
}
```

Values:

- `static`
 - `relative`
 - `absolute`
 - `fixed`
 - `sticky`
-

top, left, right, bottom

Move positioned elements.

```
div {  
  position: absolute;  
  top: 20px;  
}
```

z-index

Controls stacking order.

```
div {  
  z-index: 10;  
}
```

5. Flexbox Properties (Modern Layout)

display: flex

Enables flexbox.

```
.container {  
  display: flex;  
}
```

justify-content

Horizontal alignment.

```
justify-content: center;
```

align-items

Vertical alignment.
align-items: center;

flex-direction

Layout direction.
flex-direction: row;

Values: row, column

gap

Space between items.
gap: 20px;

6. Grid Properties

display: grid

```
.container {  
  display: grid;  
}
```

grid-template-columns

grid-template-columns: 1fr 1fr 1fr;

grid-gap / gap

gap: 10px;

7. Visibility & Effects

opacity

Transparency level.
div {
 opacity: 0.5;
}

visibility

Show or hide element.
visibility: hidden;

box-shadow

Adds shadow.
div {
 box-shadow: 0 4px 10px gray;
}

border-radius

Rounded corners.

```
div {  
  border-radius: 10px;  
}
```

8. Animation & Transition

transition

Smooth change effect.

```
button {  
  transition: 0.3s;  
}
```

transform

Move/rotate/scale elements.

```
div {  
  transform: scale(1.2);  
}
```

animation

Runs keyframe animation.

```
animation: slide 2s infinite;
```

9. Responsive Design

@media

Applies styles for screen sizes.

```
@media (max-width: 600px) {  
  body {  
    background: red;  
  }  
}
```

MOST IMPORTANT PROPERTIES (Learn First)

If you're starting, focus on:

display
margin & padding
width & height
flexbox
position
background
font & text properties

These alone control **80% of real-world UI design**.